



Universidad Argentina de la Empresa
Facultad de Ingeniería y Ciencias Exactas

Ingeniería de Datos II

Profesora: Doctora Ingeniera Roxana Martínez

Integrantes:

Perez Leal, Agustín - 1189636

Martinez Zorzi, Facundo - 1193730

Masare, Candela - 1185770

Mioni Bonuccelli, Franco Brunello - 1189642

Smith, Pedro - 1195404

Villaverde, Pilar - 1186142

Introducción

El presente documento describe tres propuestas de sistemas de software reales, cada una diseñada para justificar el uso de cuatro motores de bases de datos distintos: MongoDB, Neo4j, Redis y Apache Cassandra. La premisa central es que ningún motor único resuelve de forma óptima todos los problemas de persistencia en un sistema moderno. La elección debe estar guiada por la naturaleza del dato y la operación que lo involucra.

En las tres propuestas, MongoDB y Neo4j son los motores implementados en código Python. Redis y Cassandra se documentan conceptualmente con su justificación técnica, esquemas y comandos representativos.

Idea 1 — StreamFlow: Plataforma de Streaming de Video

Un servicio de streaming estilo Netflix que gestiona catálogo audiovisual, sesiones de usuarios, historial de reproducciones y recomendaciones personalizadas basadas en comportamiento colectivo.

Descripción general

StreamFlow es una plataforma de video on demand donde los usuarios pueden explorar un catálogo heterogéneo (películas, series, documentales), reproducir contenido, retomar donde lo dejaron y recibir recomendaciones personalizadas. Cada componente del sistema tiene una naturaleza de dato y un patrón de acceso distinto, lo que lo convierte en un caso ideal para arquitectura multi-motor.

Motores y justificación

MongoDB — Catálogo de contenido

El catálogo es el caso ideal para un motor documental porque cada tipo de contenido tiene estructura variable. Una película tiene director y duración total. Una serie tiene temporadas, episodios y showrunner. Un documental tiene temática y fecha de estreno específicas. En una base relacional se necesitarían múltiples tablas con campos NULL masivos. MongoDB permite que cada documento refleje exactamente la estructura del contenido sin imponer un esquema rígido.

Operaciones típicas:

- Búsquedas por género, año o clasificación
- Agregaciones para obtener el Top 10 mejor calificado por género
- Actualización de calificación promedio cuando un usuario puntúa
- Inserción de nuevos títulos con estructura variable sin migración de esquema

Implementación: Python con pymongo.

Neo4j — Motor de recomendaciones

Las recomendaciones son un problema de relaciones, no de datos aislados. La pregunta '¿qué debería ver este usuario?' se responde trayendo conexiones: este usuario vio X, otros usuarios que también vieron X vieron Y. En SQL esto requeriría múltiples JOINS sobre tablas potencialmente millonarias. Neo4j almacena las relaciones como punteros directos (index-free adjacency), por lo que recorrer conexiones no depende del tamaño total de la base.

Consulta central de recomendación:

```
MATCH (yo:Usuario)-[:VIO]->(c:Contenido)<-[:VIO]-(otro:Usuario)-[:VIO]->(rec:Contenido)
WHERE NOT (yo)-[:VIO]->(rec)
RETURN rec.titulo, COUNT(*) AS popularidad ORDER BY popularidad DESC LIMIT 5
```

Implementación: Python con el driver oficial neo4j.

Redis — Sesiones y caché en tiempo real

Redis vive en memoria RAM, respondiendo en milisegundos. En StreamFlow gestiona tres responsabilidades críticas: verificación de autenticación (token de sesión con TTL de 24 horas), progreso de reproducción (minuto exacto donde el usuario pausó) y listas de trending por región actualizadas en tiempo real mediante estructuras ZADD/ZRANGE.

Cassandra — Historial de reproducciones

Cada reproducción genera un evento: qué vio el usuario, cuándo, desde qué dispositivo, cuántos minutos. Con millones de usuarios simultáneos, el volumen de escrituras es constante y masivo. Cassandra distribuye los datos en nodos y escala horizontalmente sin degradar la escritura. La tabla se particiona por usuario_id y se ordena por timestamp DESC.

Resumen de motores

Motor	Rol en el sistema	Implementación	Dataset relacionado
MongoDB	Catálogo de películas y series	Python (pymongo)	MovieLens small (100K ratings)
Neo4j	Recomendaciones por comportamiento	Python (neo4j)	MovieLens small (usuarios + ratings)
Redis	Sesiones activas y progreso de video	Solo doc.	Sintético / generado
Cassandra	Historial de reproducciones	Solo doc.	Sintético / generado

Dataset recomendado

MovieLens small (GroupLens Research) — 100,000 ratings de 9,000 películas por 600 usuarios. Disponible en grouplens.org/datasets/movielens. Formato CSV, sin trámites de acceso, limpio y ampliamente documentado. Cubre catálogo para MongoDB y comportamiento de usuarios para construir el grafo de Neo4j.

Arquitectura del flujo de datos

Cuando un usuario hace una request, el sistema responde en capas:

- Redis → ¿Está autenticado? ¿En qué minuto pausó? (milisegundos)
- MongoDB → ¿Qué información tiene ese título del catálogo? (consulta flexible)
- Cassandra → Registrar el evento de reproducción (escritura masiva)
- Neo4j → ¿Qué contenido le recomendamos a continuación? (traversal de grafo)

Idea 2 — EduTrack: Plataforma de Aprendizaje Online

Una plataforma de e-learning donde los alumnos acceden a cursos con contenido heterogéneo, avanzan por rutas de aprendizaje con prerequisites, y el sistema registra cada interacción para analítica educativa.

Descripción general

EduTrack es un sistema educativo online donde conviven cursos con estructura muy variada: algunos tienen videos, quizzes y proyectos; otros son solo lectura y evaluación final. Los cursos tienen prerequisites entre sí (un grafo de dependencias), los alumnos generan eventos continuos de actividad, y el sistema debe responder rápido a consultas de progreso en tiempo real.

Motores y justificación

MongoDB — Catálogo de cursos y contenido educativo

El contenido educativo es intrínsecamente heterogéneo: un curso de programación tiene ejercicios con casos de prueba y criterios de aceptación. Un curso de diseño tiene galerías y rúbricas de evaluación visual. Un curso de matemática tiene fórmulas y ejercicios procedurales. Forzar un esquema relacional implicaría decenas de tablas con NULLs masivos. MongoDB permite que cada documento de curso tenga exactamente los campos que necesita su naturaleza.

Operaciones típicas:

- Búsqueda de cursos por categoría, nivel o duración estimada
- Recuperación del contenido completo de un módulo (videos, lecturas, ejercicios)
- Actualización del rating promedio del curso cuando un alumno lo evalúa
- Inserción de nuevos tipos de recurso sin alterar el esquema existente

Implementación: Python con pymongo.

Neo4j — Grafo de prerequisites y rutas de aprendizaje

Los prerequisites entre cursos forman un grafo dirigido acíclico (DAG). Las preguntas clave son de traversal: '¿Cuál es el camino más corto para que este alumno llegue al curso X dados los cursos que ya aprobó?', '¿Qué cursos se desbloquean si aprueba Y?', '¿Qué alumnos están a un solo prerequisite de poder inscribirse en Z?'. En SQL estas consultas implicarían JOINS recursivos cuya complejidad crece con la profundidad del árbol. En Neo4j son consultas de profundidad variable nativas.

Consulta central de ruta de aprendizaje:

```
MATCH path = shortestPath(  
  (alumno:Alumno {id: $uid})-[:APROBÓ*]->(objetivo:Curso {nombre: $curso})  
)  
RETURN [n IN nodes(path) | n.nombre] AS ruta
```

Implementación: Python con el driver oficial neo4j.

Redis — Sesiones activas y progreso de lección

Durante una sesión activa, el sistema necesita saber en qué segundo del video está el alumno, si está en medio de un quiz (y cuántas preguntas respondió), y cuánto tiempo lleva en la sesión actual. Estos datos son volátiles, cambian constantemente y necesitan respuesta en milisegundos. Consultarlos desde MongoDB o Cassandra en cada interacción sería innecesariamente costoso.

Cassandra — Registro de actividad y learning analytics

Cada click, cada respuesta a un quiz, cada pausa en un video, cada vez que el alumno abre o cierra una lección genera un evento. Estos eventos alimentan analíticas de aprendizaje (learning analytics) que permiten detectar en qué punto del curso los alumnos abandonan, qué preguntas generan más errores, y qué contenidos tienen mayor engagement. El volumen de escritura es continuo y masivo, exactamente el caso para el que Cassandra fue diseñada.

Resumen de motores

Motor	Rol en el sistema	Implementación	Dataset relacionado
MongoDB	Catálogo de cursos y recursos educativos	Python (pymongo)	OULAD (Open University)
Neo4j	Prerequisitos y rutas de aprendizaje	Python (neo4j)	OULAD (cursos y relaciones)
Redis	Sesión activa y progreso de lección	Solo doc.	Sintético / generado
Cassandra	Eventos de actividad del alumno	Solo doc.	OULAD (interacciones VLE)

Dataset recomendado

Open University Learning Analytics Dataset (OULAD) — Dataset real de una universidad británica con alumnos, cursos, resultados académicos e interacciones con el entorno virtual de aprendizaje (VLE). Disponible en Kaggle y en el sitio oficial de la Open University. Sin trámites de acceso, cubre los cuatro motores de forma natural: catálogo para MongoDB, relaciones entre cursos para Neo4j, e interacciones masivas para Cassandra.

Arquitectura del flujo de datos

Cuando un alumno interactúa con el sistema:

- Redis → ¿Dónde estaba en el video? ¿Cuántas preguntas del quiz respondió? (milisegundos)
- MongoDB → ¿Qué contenido tiene este módulo? ¿Cuáles son los recursos? (consulta flexible)
- Cassandra → Registrar el evento de interacción para analítica (escritura masiva)
- Neo4j → ¿Qué cursos puede tomar a continuación según sus aprobaciones? (traversal de grafo)

Idea 3 — GameNexus: Plataforma Social de Gaming

Un launcher social de gaming (estilo Steam Community) donde los jugadores gestionan su biblioteca de juegos, mantienen una red de amigos, comparten su actividad y reciben sugerencias de con quién jugar según comportamiento en común.

Descripción general

GameNexus es una plataforma que combina catálogo de juegos, red social de jugadores y registro de actividad. No gestiona descargas ni instalaciones (eso lo hace el SO), sino que se focaliza en los datos: qué juega cada usuario, con quién, cuándo y qué logros obtuvo. Esta combinación de datos estructurados variables, relaciones sociales, estado en tiempo real y eventos masivos la hace un caso ideal para arquitectura multi-motor.

Motores y justificación

MongoDB — Catálogo de juegos y biblioteca personal

Cada juego tiene una estructura radicalmente distinta: un RPG tiene sistema de clases, árbol de habilidades y modo campaña. Un battle royale tiene estadísticas de partida, ranking de jugadores y temporadas activas. Un juego indie puede tener solo descripción, capturas y precio. Forzar un esquema relacional implicaría docenas de tablas con NULLs masivos. MongoDB permite que cada documento refleje exactamente las características del juego. Además, la biblioteca personal del usuario (qué juegos tiene, horas jugadas por título, logros desbloqueados) se puede embeber como subdocumento en el perfil del usuario.

Operaciones típicas:

- Búsqueda de juegos por género, plataforma o rango de precio
- Recuperación del perfil completo de un jugador con su biblioteca y logros
- Actualización de horas jugadas y logros obtenidos al cerrar sesión
- Inserción de nuevos juegos con estructura variable sin alterar el esquema

Implementación: Python con pymongo.

Neo4j — Red social de jugadores

Las preguntas centrales de GameNexus son de relaciones: '¿Quiénes son los amigos de mis amigos que juegan el mismo título que yo?', '¿Con quién debería jugar una partida esta noche?', '¿Qué jugadores de mi red tienen logros que yo no tengo en un juego que comparto con ellos?'. En SQL estas consultas implicarían JOINS recursivos sobre tablas de millones de filas. En Neo4j son traversals directas.

Punto no obvio: Neo4j y Redis están más acoplados que en las otras ideas. Cuando el sistema sugiere con quién jugar, Neo4j trae los amigos candidatos por afinidad de juegos, y Redis confirma cuáles están efectivamente online en ese momento. Son dos motores consultados en secuencia para responder una sola pregunta.

Consulta central de sugerencia de compañeros:

```
MATCH (yo:Jugador {id: $uid})-[:AMIGO]->(amigo:Jugador)-[:JUEGA]->(j:Juego)
<-[:JUEGA]-(yo)
WHERE amigo.id IN $onlineIds
RETURN amigo.nombre, j.titulo, COUNT(j) AS afinidad ORDER BY afinidad DESC
```

Implementación: Python con el driver oficial neo4j.

Redis — Estado en tiempo real y leaderboards en vivo

Redis resuelve tres problemas simultáneos que necesitan respuesta en milisegundos: estado online del jugador (si está jugando, en qué juego, en qué sala), estado de la partida en curso (jugadores conectados, puntaje parcial, tiempo restante), y leaderboards en vivo actualizados mediante ZADD/ZRANGE sin consultar Cassandra. Es el motor más crítico durante una partida activa: si cae, el jugador pierde su sesión; si tarda, el lag es perceptible.

Cassandra — Historial de partidas y estadísticas

Cada partida finalizada genera un registro: quién jugó, cuándo, resultado, kills, duración, XP ganado, ítems usados. Con millones de jugadores activos simultáneos, esto es escritura masiva y continua. Cassandra particiona por jugador_id y ordena por timestamp DESC, permitiendo consultar el historial reciente de cualquier jugador eficientemente sin degradar el rendimiento de escritura.

Resumen de motores

Motor	Rol en el sistema	Implementación	Dataset relacionado
MongoDB	Catálogo de juegos y biblioteca del usuario	Python (pymongo)	Steam Games Dataset (Kaggle)
Neo4j	Red social y sugerencias de compañeros	Python (neo4j)	Steam User Data (Kaggle)
Redis	Estado online y leaderboards en vivo	Solo doc.	Sintético / generado
Cassandra	Historial de partidas y estadísticas	Solo doc.	Sintético / generado

Dataset recomendado

Combinación de Steam Games Dataset + Steam User Data (Kaggle). El primero provee metadata de juegos (géneros, plataformas, ratings, descripción) para poblar MongoDB. El segundo incluye comportamiento de usuarios (juegos en biblioteca, horas jugadas por título) desde el cual se infieren conexiones entre jugadores para construir el grafo de Neo4j. Ambos son descargables directamente desde Kaggle sin trámites. Nota: ningún dataset de gaming incluye relaciones sociales reales entre usuarios; el grafo de Neo4j se construye inferiendo afinidad por juegos en común.

Arquitectura del flujo de datos

Cuando un jugador interactúa con la plataforma:

- Redis → ¿Está online? ¿En qué partida? ¿Cuál es su posición en el ranking live? (milisegundos)
- MongoDB → ¿Qué información tiene ese juego? ¿Cuáles son sus logros? (consulta flexible)
- Cassandra → Registrar resultado de la partida finalizada (escritura masiva)
- Neo4j → ¿Con quién de su red debería jugar? ¿Quiénes juegan lo mismo? (traversal de grafo)

Conclusión comparativa

Las tres propuestas comparten la misma lógica de asignación de motores: MongoDB para datos heterogéneos con estructura variable, Neo4j para relaciones y traversals, Redis para estado volátil en tiempo real, y Cassandra para escritura masiva y continua. La diferencia está en el dominio y en cómo cada motor se justifica desde la naturaleza real del dato en ese contexto.

	StreamFlow	EduTrack	GameNexus
MongoDB para	Catálogo audiovisual	Cursos heterogéneos	Juegos + biblioteca
Neo4j para	Recomendaciones	Prerequisitos	Red social
Redis para	Sesión + progreso	Sesión + quiz activo	Estado online + ranking
Cassandra para	Historial de reproduc.	Eventos de actividad	Historial de partidas
Dataset	MovieLens small	OULAD	Steam (Kaggle)